Gradle, you can build the container image by running this command:

```
$ docker build -t catalog-service .
```

If you're using Maven, you can specify the location used by Maven as a build argument with the following command (don't forget the final dot):

```
$ docker build --build-arg JAR_FILE=target/*.jar -t catalog-service .
```

In either case, you'll end up with your Catalog Service application packaged as a container image. Since we haven't specified any version, the image will be tagged as `latest` automatically. Let's verify that it works.

Remember the two aspects I covered in the previous section: port forwarding and using the Docker built-in DNS server. You can handle them by adding two arguments to the `docker run` command:

- `-p 9001:9001` will map port 9001 inside the container (where the Catalog Service is exposing its services) to port 9001 on your localhost.
- `--net catalog-network` will connect the Catalog Service container to the catalog-network you previously created so that it can contact the PostgreSQL container.

That is still not enough. In the previous chapter, we set the `spring.datasource.url` property for Catalog Service to `jdbc:postgresql://localhost:5432/polardb_catalog`. Since it points to `localhost`, it will not work from within a container. You already know how to configure a Spring Boot application from the outside without having to recompile it, right? An environment variable will do. We need to overwrite the